



Introduction to Linux OS

By Mohamed Gamal



[Comparison of Operating Systems](#)

[List of Linux Distributions](#)

[Comparison of Linux Distributions](#)

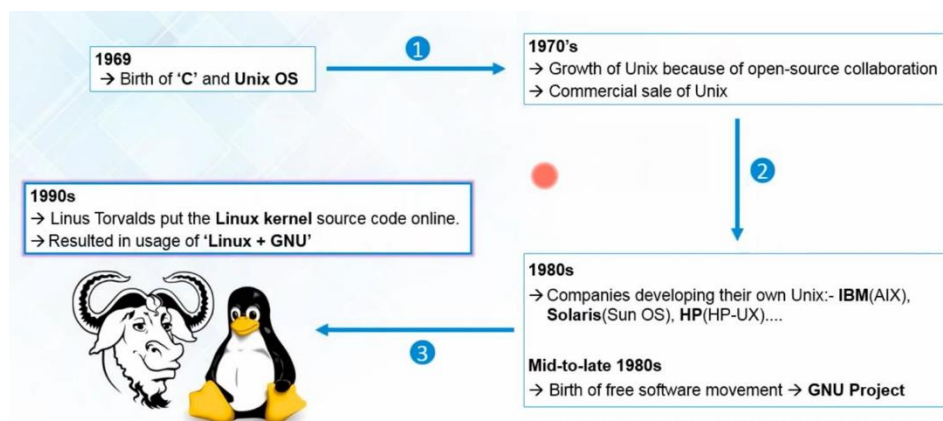
Linux History (What, Who, When, Where & Why Linux)

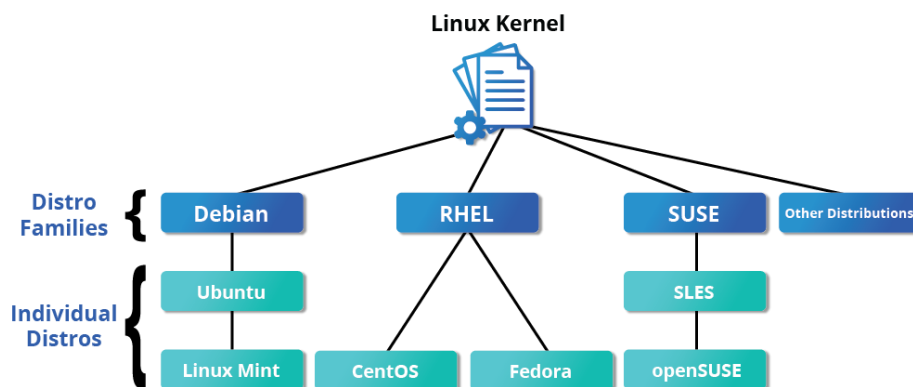
[Linux](#) is an open source computer operating system, initially developed on and for Intel x86-based personal computers. It has been subsequently ported to an astoundingly long list of other hardware platforms, from tiny embedded appliances to the world's largest supercomputers.



Linus Torvalds was a student in Helsinki, Finland, in 1991, when he started a project: writing his own operating system kernel. He also collected together and/or developed the other essential ingredients required to construct an entire operating system with his kernel at the center using C programming language. It wasn't long before this became known as the Linux kernel.

In 1992, Linux was re-licensed using the [General Public License \(GPL\)](#) by GNU (a project of the Free Software Foundation or FSF, which promotes freely available software), which enabled it to build a worldwide community of developers. By combining the kernel with other system components from the GNU project, numerous other developers created complete systems called Linux Distributions, which first appeared in the mid-90s.

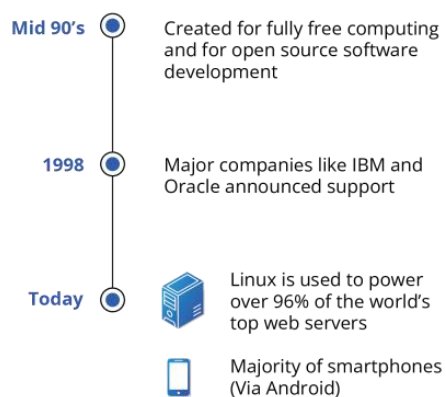




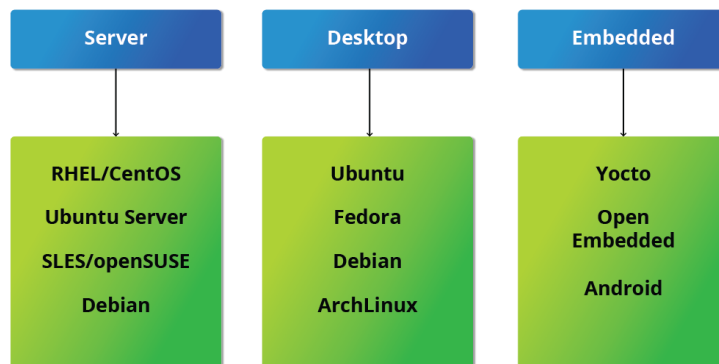
RHEL: [Red-Hat Enterprise Linux](#)

SUSE: Software und System-Entwicklung (i.e., Software and System-Design)

SUSE: Linux Enterprise Server



Choosing a Linux Distribution



What will we use? We'll use [CentOS Stream 9](#) which is a RHEL based distribution.

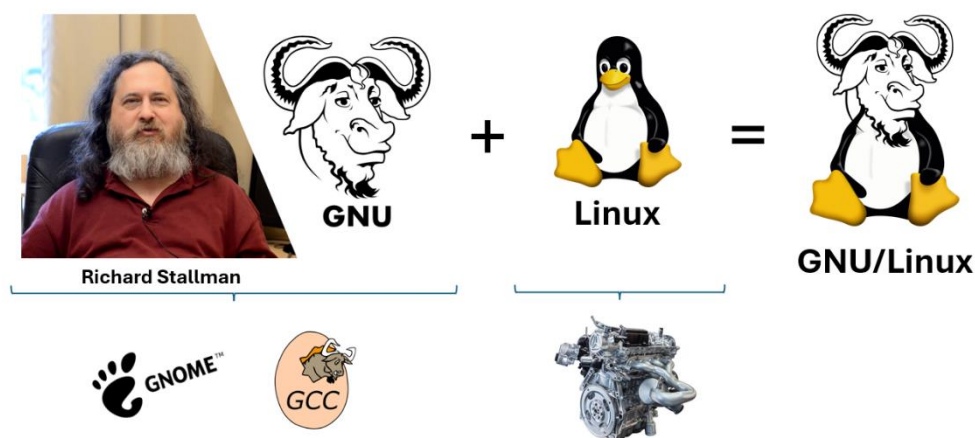


GNOME Desktop Environment

- [GNOME \(GNU Network Object Model Environment\)](#) is a popular desktop environment with an easy-to-use graphical user interface used on various Unix-like operating systems, such as Linux distributions.
- It is bundled as the default desktop environment for most Linux distributions, including Red Hat Enterprise Linux (RHEL), Fedora, CentOS, SUSE Linux Enterprise, Ubuntu, and Debian.
- GNOME has menu-based navigation and is sometimes an easy transition to accomplish for Windows users.
- However, the look and feel can be quite different across distributions, even if they are all using GNOME.

Another common desktop environment very important in the history of Linux and also widely used is KDE (K Desktop Environment), which has often been used in conjunction with SUSE and openSUSE.

Other alternatives for a desktop environment include Unity (present on older Ubuntu but still based on GNOME), XFCE, and LXDE.

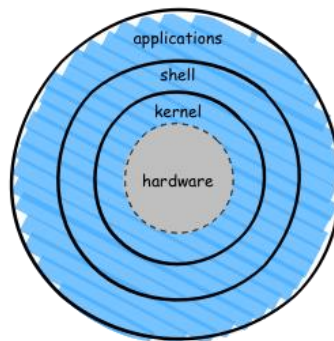




Introduction to the Command Line

Linux system administrators spend a significant amount of their time at the command line prompt. They often automate and troubleshoot tasks in this text environment. There is a saying, "graphical user interfaces make easy tasks easier, while command line interfaces make difficult tasks possible". Linux relies heavily on the abundance of command line tools. The command line interface provides the following advantages:

- No GUI overhead is incurred.
- Virtually any and every task can be accomplished while sitting at the command line.
- You can implement scripts for often-used (or easy-to-forget) tasks and series of procedures.
- You can sign into remote machines anywhere on the Internet.
- You can initiate graphical applications directly from the command line instead of hunting through menus.
- While graphical tools may vary among Linux distributions, the command line interface does not.



The Command Line

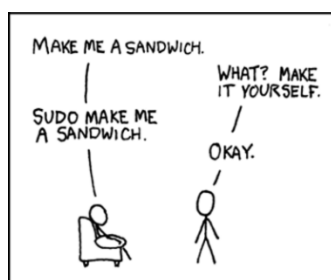
Most input lines entered at the shell prompt have three basic elements:

```
$ <command> <options> <arguments>
```

The command is the name of the program or script you are executing. It may be followed by one or more options (or switches) that modify what the command may do. Options usually start with one or two dashes, for example, -p or --print, in order to differentiate them from arguments, which represent what the command operates on. However, plenty of commands have no options, no arguments, or neither.

Sudo (SuperUser Do)

All the demonstrations created have a user configured with sudo capabilities to provide the user with administrative (admin) privileges when required. sudo allows users to run programs using the security privileges of another user, generally root (superuser).





Command	Usage
su	Login as a super user with high privileges (use 'exit' or 'CTRL + D' to exit sudo mode).
man	Displays the manual page for a command, providing detailed information on its usage. <code>\$ man diff</code>
init 6	Reboots (restarts) the system.
reboot	
init 0	Shuts down the system.
shutdown	
who	Shows who is currently logged into the system.
whoami	Displays the username of the current user.
last	Displays a list of last logged in users.
uname	Prints system information.
hostname	Display or set the hostname of the system (<code>\$ cat /etc/hostname</code>).
hostnamectl	A more comprehensive way to manage the system hostname and related metadata.
history	Shows the command history of the current session.
passwd	Change the current or other user's password.
reset	Resets the terminal.
clear	Clears the terminal screen.
echo	Prints a string in the terminal screen. <code>\$ echo Hello</code> <code>\$ echo -e "Ahmed\nMohamed\nMona"</code>

Hints:

- The **`tab`** button autocompletes commands and filenames/directories.
- Dealing with Long Shell Commands: the backslash (\) is the line continuation character.

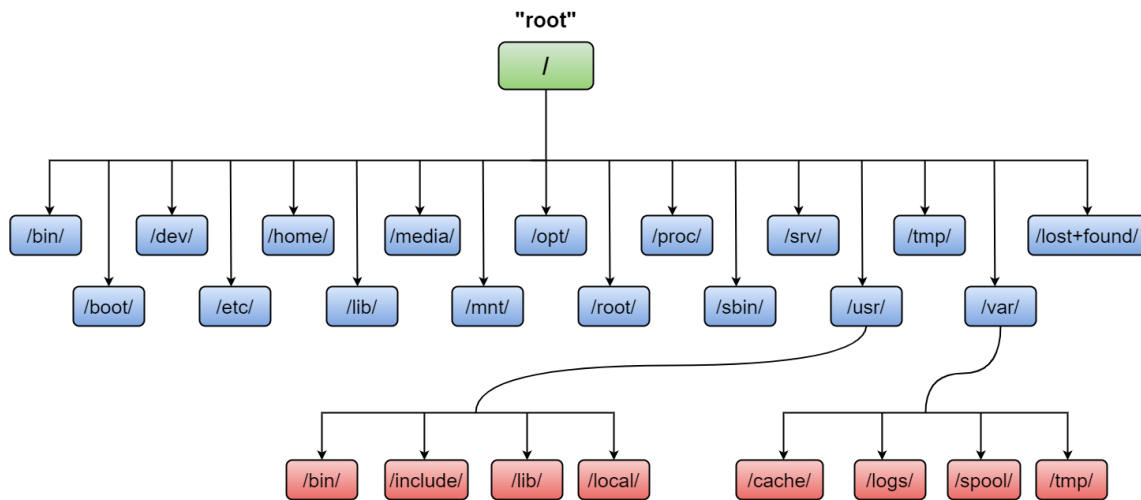
```
$ echo "one \  
two"
```



Exploring the Filesystem

The file structure is nothing but how an individual computer organizes its data and stores it. Linux file structure is somehow different than the Windows and Macintosh operating systems.

In the Linux operating system Filesystem Hierarchy Standard (FHS), all files and directories appear under the “root” (/) directory, even if they are stored on different physical or virtual devices.



root (UID 0)



Dir	Description
/	The directory called "root." It is the starting point for the file system hierarchy. Note that this is not related to the root, or superuser, account.
/bin	Binaries and other executable programs.
/etc	System configuration files.
/home	Home directories.
/opt	Optional or third party software.
/tmp	Temporary space, typically cleared on reboot.
/usr	User related programs.
/var	Variable data, most notably log files.

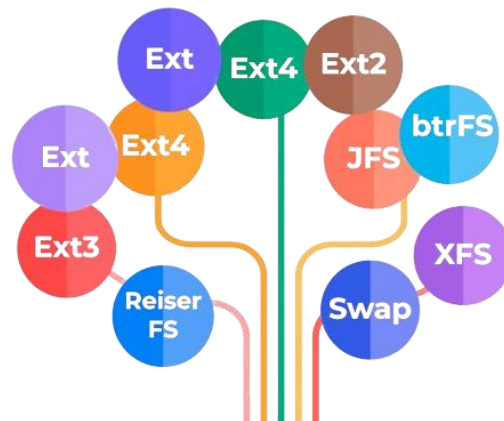


Comprehensive Directory Listing

Dir	Description
/	The directory called "root." It is the starting point for the file system hierarchy. Note that this is not related to the root, or superuser, account.
/bin	Binaries and other executable programs.
/boot	Files needed to boot the operating system.
/cdrom	Mount point for CD-ROMs.
/cgroup	Control Groups hierarchy.
/dev	Device files, typically controlled by the operating system and the system administrators.
/etc	System configuration files.
/export	Shared file systems. Most commonly found on Solaris systems.
/home	Home directories.
/lib	System Libraries.
/lib64	System Libraries, 64 bit.
/lost+found	Used by the file system to store recovered files after a file system check has been performed.
/media	Used to mount removable media like CD-ROMs.
/mnt	Used to mount external file systems.
/opt	Optional or third party software.
/proc	Provides information about running processes.
/root	The home directory for the root account.
/sbin	System administration binaries.
/selinux	Used to display information about SELinux.
/srv	Contains data which is served by the system.
/srv/www	Web server files.
/srv/ftp	
/sys	Used to display and sometimes configure the devices and busses known to the Linux kernel.
/tmp	Temporary space, typically cleared on reboot. This directory can be used by the OS and users alike.
/usr	User related programs, libraries, and documentation. The sub-directories in /usr relate to those described above and below.
/usr/bin	Binaries and other executable programs.
/usr/lib	Libraries.
/usr/local	Locally installed software that is not part of the base operating system.
/usr/sbin	System administration binaries.
/var	Variable data, most notably log files.
/var/log	Log files.



Types of Linux File Systems:



File Naming in Linux:

- In any Linux-based system, you do not have to specify the file extension when creating a file.
- However, file extensions are useful for identifying the type of content or for associating the file with the appropriate application.
- File names may be up to 255 characters.
- Avoid special characters as > < ? * # ' .
- File names are case sensitive (e.g., file1 ≠ FILE1)

Listing All User Commands

To list all the commands available to the user in the current shell session you can use

```
$ compgen -c    or    $ man -k .
```



Locating Applications & Commands (WH Commands)

Depending on the specifics of your particular distribution's policy, programs and software packages can be installed in various directories. In general, executable programs and scripts should live in the **/bin**, **/usr/bin**, **/sbin**, **/usr/sbin** directories, or somewhere under **/opt**. They can also appear in **/usr/local/bin** and **/usr/local/sbin**, or in a directory in a user's account space, such as **/home/mg/bin**.

One way to locate programs/commands is to employ the **which** utility.

Example Usage	
<pre>\$ which firefox /usr/bin/firefox</pre>	<pre>\$ which cat /usr/bin/cat</pre>

If **which** does not find the program, **whereis** is a good alternative because it looks for packages in a broader range of system directories:

Example Usage	
<pre>\$ whereis firefox firefox: /usr/bin/firefox /usr/lib64/firefox /usr/share/man/man1p/firefox.1p.gz</pre>	
<pre>\$ where is cat cat: /usr/bin/cat /usr/share/man/man1/cat.1.gz /usr/share/man/man1p/cat.1p.gz</pre>	



Environment Variables

The **PATH** environment variable contains a list of directories that contain executable commands.

```
$ echo $PATH
```

When you type in a command at the prompt and press Enter, that command will be searched for in the directories in your **\$PATH**.

For example, **/bin** will be searched first, if the command is found it will be executed. If it is not found, then **/usr/bin** will be searched and so on.

If no executable command is found that matches your request, you will be politely told that it cannot be found.

Common Environment Variables

```
$ env or $ printenv
```

Variable	Description
EDITOR	The program to run to perform edits
HOME	The Home directory of the user
LOGNAME	The login name of the user.
MAIL	The location of the user's local inbox
OLDPWD	The previous working directory.
PATH	A colon separated list of directories to search for commands.
PAGER	This program may be called to view a file.
PS1	The primary prompt string.
PS2	The secondary prompt string.
PWD	The current working directory
USER	The username of the user.

Shell Customization:

```
PS1="<\t \u@\h \w>\$ "
```

```
PS2="line continued: "
```

[Link 1](#) [Link 2](#)

Symbol	Description
\t	The current time in 24-hour HH:MM:SS format
\u	The username of the current user
\h	The hostname up to the first '
\w	The current working directory, with \$HOME abbreviated with a tilde
\\$	If the effective UID is 0, a #, otherwise a \$
...	...

Extra: [Oh My Posh](#)



Accessing Directories (Folders)

When you first log into a system or open a terminal, the default directory should be your home directory. You can see the exact location by typing `echo $HOME`.

Directories are simply containers for files and other directories. They provide a tree like structure for organizing the system. Directories can be accessed by their name and they can also be accessed using a couple of shortcuts.

Linux uses the symbols `.` and `..` to represent directories.

- Think of `.` as "this directory"
- and `..` and "the parent directory"

Symbol	Description	Examples
<code>.</code>	This (or current) directory.	<pre>\$ pwd \$ cd /var/tmp \$ cd . \$ cd ..</pre>
<code>..</code>	The parent directory.	
<code>/</code>	Directory separator.	

--- Navigating and Managing Directories ---

Command	Usage
<code>pwd</code>	Displays the current working directory.
<code>cd</code>	Change to another directory. e.g. , <code>pushd /tmp</code> <code>popd</code>
<code>pushd</code>	
<code>popd</code>	
<code>xdg-open .</code>	Open the current working directory in a graphical user interface (GUI) file manager.
<code>ls</code> or <code>ll</code>	List the contents of the current working directory.
<code>tree</code>	Displays a tree view of the filesystem.



--- Viewing and Managing Files ---

Command	Usage
touch	Creates an empty file or updates the timestamp of an existing file.
cat	Used for viewing files that are not very long; it does not provide any scroll-back.
tac	Used to look at a file backwards, starting with the last line.
less	Used to view larger files because it is a paging program. It pauses at each screen full of text, provides scroll-back capabilities, and lets you search and navigate within the file.
more	Used to view the contents of a file one screen at a time. It pauses after displaying each page of text but has limited navigation capabilities compared to `less`. You can scroll forward but not backward, making it useful for quick file viewing.
head	Used to print the first 10 lines of a file by default. You can change the number of lines by doing -n 15 or just -15 if you wanted to look at the last 15 lines instead of the default.
tail	The opposite of head; by default, it prints the first 10 lines of a file. Use <code>\$ tail -f <file></code> to view changes that are being made in real time to a file.
wc	Displays the number of lines, words, and characters in a file.
stat	Used to display detailed information about files and directories.
file	Used for determining the type of a file by examining its content rather than the file extension. <code>\$ file *</code> <code>\$ file script.sh</code>

--- Creating, Managing, and Removing Files and Directories ---

Command	Usage
mkdir	Creates a directory under the current directory.
rmdir	Removes an <u>empty</u> directory.
rm	Removes files or directories. Use the -r option to remove a directory and its contents recursively.
cp	Copies files or directories.
mv	Moves or renames files or directories. <code>\$ mv file1.txt /home</code> <code>\$ mv file1.txt file2.txt file3.txt /home</code> <code>\$ mv file1.txt file1_renamed.txt</code>
du -ah	Estimates and displays the disk space usage of files and directories.
!!	Run the last recent command.



File Globbing (Meta Character)

- **Asterisk (*)**: represents 0 or more characters, except leading (.)
- **Question mark (?)**: represents any single character except the leading (.)

```
$ touch file{1..3}.txt
```

```
$ ls file?.txt
```

```
$ ls fileB?.txt
```

```
$ ls file.t?t
```

```
$ ls -d dir?
```

```
$ cp file?.txt /backup
```



Finding Files – find Command

What if you want to locate a file or directory? The first command you can use is the **find** command. It can be used to find files by name, size, permissions, owner, modification time, and more.

`$ find [path] [expression]`

Command	Usage	Example
<code>find path -name pattern</code>	Displays files/directories whose name matches pattern. This is case sensitive.	<pre>\$ sudo find / -name firefox \$ find /usr/local -name *conf \$ find . -name "*.sh"</pre>
<code>find path -iname pattern</code>	Same as -name but ignores case.	<pre>\$ find . -iname "log*" \$ find /var -iname apache</pre>
<code>find path -regex pattern</code>	Search using regular expressions .	<pre>\$ find . -regex ".*\.(txt log)\$"</pre>
<code>find path -type d f</code>	Find only files or directories.	<pre>\$ find / -type d -name log \$ find . -type f -name "pkc*" \$ find . -type d -name firefox</pre>
<code>find path -ls</code>	Performs an ls on each of the found files or directories.	<pre>\$ find . -ls \$ find / -name "*.sh" -ls</pre>
<code>find path -mtime [- +]⁻days</code> - Last + Exactly + More than	mtime: modified time Finds files that are num_days old.	<pre>\$ find . -mtime -7 \$ find . -mtime 5 \$ find . -mtime +30 \$ find . -mtime +10 -mtime -13</pre> Files that are more than 10 days old, but less than 13 days old.
<code>find path -size [- +]⁻size</code>	Finds files that are of size less/greater/equal to num.	<pre>\$ find . -size -5k \$ find . -size 4G \$ find . -size +3M \$ find . -size +1M -mtime -3M</pre>
<code>find path -user name</code>	Find files owned by a specific user.	<pre>\$ find /home -user MG</pre>
<code>find path -newer file</code>	Finds files that are newer than file (i.e., modified after).	<pre>\$ find . -newer log.txt</pre>
<code>find path -exec command {} \;</code>	Run command against all the files that are found.	<pre>\$ find . -type f -exec file {} \; \$ find . -name "*.txt" -exec rm {} \; \$ find . -name "*.log" -exec mv {} /var/logs/ \;</pre>

Understanding the Output of -ls flag

131	4	rw- r-- r--	1	centos9	centos9	56	Aug 5	16:52	file.py
↓	↓	↓	↓	↓	↓	↓	↓	↓	↓
Inode Number (unique identifier)	Block Count (no. 512-byte blocks)	Perm	No. hard links	Owner/User	Group	Size	Date	Time	Name



Finding Files – locate Command

`$ locate f*`

Command	Usage	Example
<code>\$ locate name</code>	Displays files with given name. This is case sensitive.	<code>\$ locate firefox</code> <code>\$ locate config.json</code> <code>\$ locate log</code>
<code>\$ locate -i name</code>	Case-insensitive search.	<code>\$ locate -i FIREFOX</code>
<code>\$ locate -r pattern</code>	Search using regular expressions.	<code>\$ locate -r "\.txt\$ \.log\$"</code>
<code>\$ locate -c name</code>	Count the number of matching results.	<code>\$ locate -c firefox</code>

find vs. Locate

Feature	`find` Command	`locate` Command
Search Method	Real-time traversal	Pre-built database (mlocate.db) <code>\$ sudo cat /var/lib/mlocate/mlocate.db</code> <code>\$ sudo updatedb</code>
Speed	Slower	Faster
Accuracy	Always up-to-date	May be outdated (unless <code>\$ updatedb</code> ran)
Dependencies	None	Requires <code>mlocate</code> and <code>updatedb</code>
Customization	Highly customizable	Limited
Best For	Complex searches, real-time data	Quick name-based searches



Comparing Files

If you want to compare two files and display the differences you can use `diff`, `sdiff`, or `vimdiff`.

file1.txt	file2.txt
Hello World This is a test file. It contains some sample text. Let's compare these files using diff. The quick brown fox jumps over the lazy dog.	Hello Linux This is a test file. It contains some modified text. Let's compare these files using the diff command. The quick brown fox leaps over the lazy cat.

\$ diff file1 file2

Example	Description
<code>\$ diff file1.txt file2.txt</code>	Line-by-Line Comparison. < indicates content from file1.txt > indicates content from file2.txt XcY → Change (c) from line(s) X in file1.txt to line(s) Y in file2.txt.
<code>\$ diff -y file1.txt file2.txt</code>	Side-by-Side Comparison. The symbol means the line is different between the two files. Use <code>--suppress-common-lines</code> flag to show only different lines.
<code>\$ diff -i file1.txt file2.txt</code>	Ignore case differences.
<code>\$ diff -r dir1 dir2</code>	Recursively compares all files inside dir1 and dir2.

\$ sdiff file1 file2

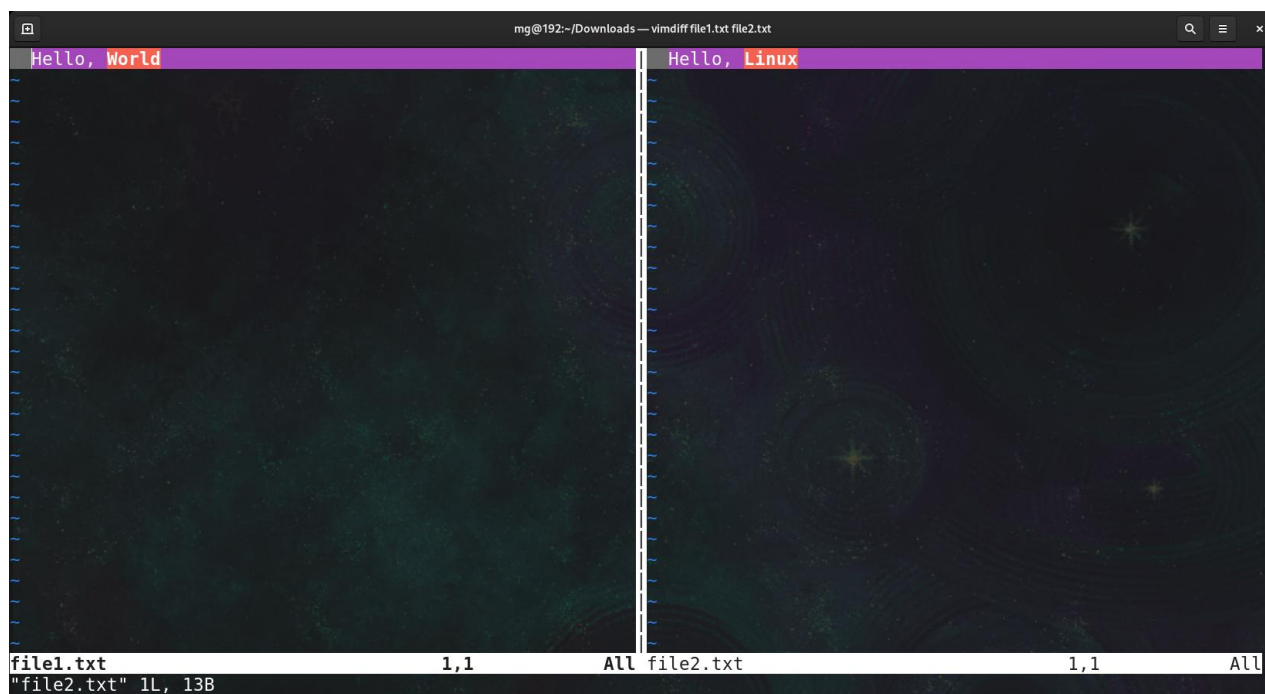
Example	Description
<code>\$ sdiff file1.txt file2.txt</code>	Side-by-Side Comparison. Use either <code>--suppress-common-lines</code> or <code>-s</code> flags to show only different lines.
<code>\$ sdiff -i file1.txt file2.txt</code>	Ignore case differences.
<code>\$ sdiff -o merged.txt file1.txt file2.txt</code>	Automatically merge two files.



```
$ vimdiff file1 file2
```

Highlight the differences between two files in the vim editor

Action	Command
Switch windows	Ctrl + w + w/↵
Next difference	]c
Previous difference	[c
Get change from right	:diffget or do
Put change to right	:diffput or dp
Recalculate differences	:diffupdate
Quit Vimdiff	:q or :qa





Sorting Files

In the simplest form the `sort` command sorts lines of text alphabetically.

file.txt	data.csv
Hello World This is a test file. It contains some sample text. It contains some sample text. Let's sort this file using sort. The quick brown fox jumps over the lazy dog.	1, Mohamed, 5000, Egypt 2, John, 3500, USA 3, Mona, 4100, Germany 4, Suhaib, 2900, Pakistan

Command	Description
<code>\$ sort file</code>	Sort text in file.
<code>\$ sort -r file</code>	Sort in reverse order.
<code>\$ sort -k F file</code>	Sort by key. The F following -k is the field/column number.
<code>\$ sort -u file</code>	Sort text in file uniquely, removing duplicate lines.

```
$ sort file.txt
$ sort -u file.txt
$ sort -ru file.txt
$ sort -u -k 2 data.csv
$ sort -k 2n data.csv
$ sort -k 2r data.csv
$ sort -t ',' -k 2 data.csv
```

Option	Description
<code>-k <field></code>	Sort by the <u>specified field</u> (e.g., <code>-k 2</code> sorts by the second field).
<code>-k <start>,<end></code>	Sort by a <u>range of fields</u> , e.g., <code>-k 2,3</code> sorts by the second and third fields.
<code>-k <field>n</code>	Sort <u>numerically</u> on the specified field, e.g., <code>-k 2n</code> sorts the second field numerically.
<code>-k <field>r</code>	Sort in <u>reverse order</u> on the specified field, e.g., <code>-k 2r</code> sorts the second field in reverse.



Searching for Text in ASCII Files

If you are looking for text within a file, use the `grep` command.

```
$ grep <pattern> <file>
```

secret.txt
site: github.com user: Mohamed pass: ABC123

Command	Usage	Example
<code>\$ grep keyword file</code>	Search for the a keyword (case-sensitive) in a file.	<code>\$ grep user secret.txt</code> <code>\$ grep o secret.txt</code>
<code>\$ grep -v keyword file</code>	Invert the match, displaying non-matching lines.	<code>\$ grep -v o secret.txt</code>
<code>\$ grep -i keyowrd file</code>	Ignoring case (case-insensitive).	<code>\$ grep -i abc secret.txt</code>
<code>\$ grep -c keyword file</code>	Count the number of matching lines. Only outputs the count, not the matching lines.	<code>\$ grep -c o secret.txt</code> <code>\$ grep -ci USER secret.txt</code>
<code>\$ grep -n keyword file</code>	Count the number of matching lines. Displays the matching lines.	<code>\$ grep -n o secret.txt</code> <code>\$ grep -ni USER secret.txt</code>



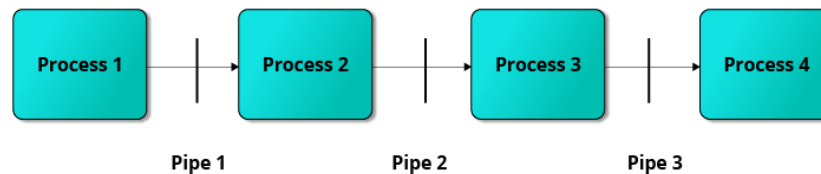
Pipes

The pipe (|) means take the standard output from the preceding command and pass it as the standard input to the following command. You can pipe the output of one command or program into another as its input.

```
$ command1 | command2 | command3 | ...
```

The above represents what we often call a **pipeline** and allows Linux to combine the actions of several commands into one.

Furthermore, there is no need to save output in (temporary) files between the stages in the pipeline, which saves disk space and reduces reading and writing from disk, which often constitutes the slowest bottleneck in getting something done.



```
$ ls -l /etc | wc -l
```

```
$ ls -lah | sort
```

```
$ ls | grep -i FILE
```

```
$ compgen -c | wc -l
```

```
$ cat file.txt | grep Mohamed | uniq
```

```
$ cat /etc/passwd | less
```

```
$ cat /etc/passwd | cut -d: -f1 | sort
```

```
$ du -h | sort -h
```

```
$ du -ah /var/log | sort -rh | head -10
```

```
$ ps aux | grep httpd
```

```
$ dmesg | tail -10 (diagnostic messages)
```

```
$ dmesg | grep -i "eth"
```



Standard Output and Error

In Linux and Unix-like operating systems the two primary channels that are part of the standard streams used for input and output operations are

- 1) **Standard Output (stdout)**
- 2) **Standard Error (stderr)**

Aspect	stdout	stderr
Purpose	Displays normal output	Displays error messages
File Descriptor	1	2
Default Output	Terminal	Terminal
Redirection	> 1> and >> ">" or "1>" to direct output by <u>overwriting</u> to the file ">>" to direct output by <u>appending</u> to the file	2>
	2>&1 or &> Redirects both stdout and stderr	

Example	Explanation
\$ echo "Hello World" > file.txt	Direct output by overwriting to the file
\$ echo "World" >> file.txt	Direct output by appending to the file
\$ ls -l /home/MG >> mg_list.txt	Direct output by appending to the file
\$ ls MG 2> err.txt	Direct error if exists to a file instead of screen
\$ ls nonexistentfile 2> /dev/null \$ ls Documents/ Downloads/ Data/ 2> /dev/null	
\$ ls MG/ Documents/ 2> err.txt 1> out.txt	Direct error to err.txt and output to out.txt
\$ find /etc -type f -name passwd > /tmp/output 2> /dev/null	Direct the standard output and the standard error
\$ find /etc -type f -name passwd >> /tmp/save-both 2>&1 \$ find /etc -type f -name passwd &> /tmp/save-both	Save both output and error to the same file 2>&1 or &>
\$ ls Documents/ MG/ &> both	

`/dev/null` is a special file that acts as a "black hole" (aka bit bucket); anything written to it is discarded, preventing error messages from being displayed on the terminal.

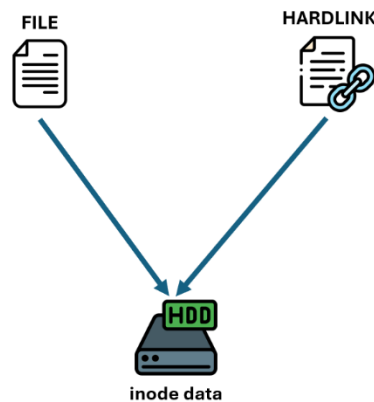


Hard and Soft (Symbolic) Links

In Linux, links are used to create references to files or directories. They can be of two types: **hard links** and **soft (symbolic) links**. Each type has specific characteristics, advantages, and use cases.

1. Hard Links

A hard link is essentially an additional name for a file. It points directly to the file's inode (the metadata of the file on disk), and multiple hard links to the same file share the same inode. It's used to provide multiple names for the same file in the same filesystem.



- **Direct Link to Data:** Hard links point to the same inode as the original file.
- **Independent:** If you delete the original file, the hard link remains and the data is still accessible.
- **Cannot Link Directories:** By default, hard links cannot be created for directories to prevent circular references and filesystem corruption (i.e., infinite loop); only for files.
- **Must Be on the Same Filesystem:** Hard links cannot span across different file systems since hard links reference the **inode** of a file, and inodes are **unique to a filesystem**.

Hard links are useful in specific scenarios where you want multiple references to the same file while maintaining data consistency.

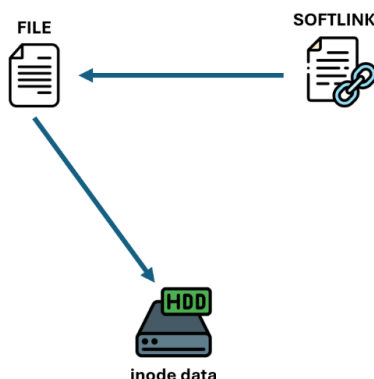
Command	Explanation
<code>\$ ln original_file hard_link_name</code>	Create a hard link
<code>\$ ls -li original_file hard_link_name</code>	Verify inode numbers

```
$ echo "Hello, World!" > original.txt
$ ln original.txt hardlink.txt
$ rm original.txt
$ cat hardlink.txt    # Data is still accessible
```



2. Soft (Symbolic) Links

A soft link is like a shortcut or alias. It contains a path to another file or directory.



- **Path-Based:** Symbolic links point to the file name (i.e., path), not the inode.
- **Breaks if the Target is Moved/Deleted:** If the original file is moved or deleted, the symbolic link becomes invalid (a "broken link").
- **Can Link Directories:** Symbolic links can point to directories.
- **Can Span Filesystems:** Symbolic links can reference files or directories on different filesystems or partitions.

Command	Explanation
\$ ln -s target_file soft_link_name	Create a symbolic link
\$ ls -li target_file soft_link_name	Identify a symbolic link

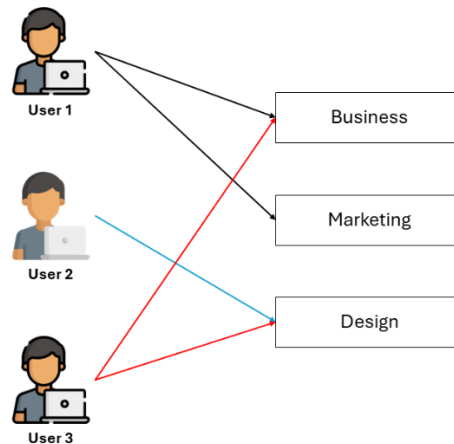
```
$ ln -s /etc/hosts hosts_link
$ cat hosts_link    # accesses the /etc/hosts file
$ cp /etc/hosts /etc/hosts.bak
$ rm /etc/hosts
$ cat hosts_link    # broken link; error since /etc/hosts is deleted
```

Feature	Hard Link	Soft Link
Works for directories?	✗ No	✓ Yes
Works across filesystems?	✗ No	✓ Yes
Breaks if original is deleted?	✗ No	✓ Yes
Shares same inode?	✓ Yes	✗ No



Linux Users and Groups

Linux/Unix operating systems have the ability to multitask in a manner similar to other operating systems. However, Linux's major difference from other operating systems is its ability to have multiple users. Linux was designed to allow more than one user to have access to the system at the same time. In order for this multiuser design to work properly, there needs to be a method to protect users from each other. This is where permissions come into play.



Linux Primary and Secondary Groups

A **primary group** is the default group that a user account belongs to when created (`/etc/group`). Every user on Linux belongs to a primary group, usually stored in the `/etc/passwd` file.

```
$ cat /etc/passwd or $ tail -n 1 /etc/passwd
```

The `/etc/passwd` file in Linux and Unix-like systems stores basic information about each user or account. It is world-readable by default and can be viewed using tools like `cat` or a text editor.

Each line in the file represents a user and contains seven fields separated by colons:

- » **Name:** The user's unique login name.
- » **Password:** Originally stored an encrypted password, now replaced by `x` (actual passwords are stored in `/etc/shadow`).
- » **User ID (UID):** A numeric identifier for the user; 0 is for root, and 100-999 are for regular users.
- » **Group ID (GID):** The user's primary group ID, usually matching the UID.
- » **GECOS:** General user information, like real name; often unused or empty.
- » **Home Directory:** The user's starting directory upon login.
- » **Shell:** The user's default shell (e.g., `/bin/bash`).

Once a user has been created with their primary group, they can be added to other **secondary groups** with a maximum of 15 secondary groups, usually stored in the `/etc/group` file.



--- Managing Users ---

Command	Description	Example
\$ adduser <username>	Create a new user without setting a password.	\$ sudo adduser Ahmed
\$ getent <entry> <username>	View a user's GECOS info.	\$ sudo getent passwd Ahmed \$ getent passwd
\$ usermod <username>	Modify a user account.	\$ sudo usermod -u 1100 Ahmed \$ sudo usermod -l newAhmed Ahmed \$ sudo usermod -d /home/New -m Ahmed
\$ userdel -r <username>	Delete a user and their home directory.	\$ sudo userdel -r Ahmed
\$ passwd <username>	Change a user's password.	\$ sudo passwd Ahmed
\$ chfn <username> \$ nano /etc/passwd	Change user information (GECOS). ▪ chfn: change finger	\$ sudo chfn Ahmed \$ sudo chfn -f "Ahmed" -o "" -p "" -h "" Ahmed
\$ su <username>	Switch to another user. ▪ su: switch user	\$ su Ahmed

--- Managing Groups ---

Command	Description	Example
\$ groupadd <group name>	Create a new group	\$ sudo groupadd Marketing
\$ groupdel <group name>	Delete (or remove) a group	\$ groupdel Marketing
\$ usermod -a -G <group name> <username>	Add a group to a user. ▪ a: append ▪ G: group	\$ usermod -a -G Marketing Ahmed
\$ gpasswd -d <username> <group name>	Remove a user from a group.	\$ gpasswd -d Ahmed Marketing
\$ groups <username>	View all groups a user in.	\$ groups Ahmed



Temporarily Switching to a Group Session

You can temporarily switch to a group session to perform some tasks, when you use the `newgrp` command, you start a new shell session where the **Primary Group** of your user is temporarily changed to the specified group. You can also exit the new shell with `exit` command or **CTRL+D**.

```
$ newgrp <group name>
```

Example

```
$ newgrp Marketing
$ id -gn      # View current active group
$ exit      # Log out of the session
```

```
mg@localhost:~/Documents
[mg@192 Documents]$ ls -l
total 0
[mg@192 Documents]$ id -gn
mg
[mg@192 Documents]$ touch file.txt
[mg@192 Documents]$ ls -l
total 0
-rw-r--r--. 1 mg mg 0 Mar 13 09:26 file.txt
[mg@192 Documents]$ newgrp admins
[mg@192 Documents]$ id -gn
admins
[mg@192 Documents]$ touch other.txt
[mg@192 Documents]$ ls -l
total 0
-rw-r--r--. 1 mg mg 0 Mar 13 09:26 file.txt
-rw-r--r--. 1 mg admins 0 Mar 13 09:27 other.txt
[mg@192 Documents]$ exit
exit
[mg@192 Documents]$ id -gn
mg
[mg@192 Documents]$
```



View all sudoers file and edit user permissions

The sudoers file is a configuration file that defines which users or groups have permission to run commands with elevated privileges (i.e., using sudo), and under what conditions, usually located at **/etc/sudoers**.

The **wheel** group is a **special administrative group** in **CentOS** (and other RHEL-based systems). Users in this group can **run commands as root** using sudo. Instead of giving root access to multiple users, you add them to wheel and control privileges via sudo.

Editing sudo Permissions for a User

Command	Description
<pre>\$ sudo -l \$ sudo -l -U <username></pre>	List sudo Privileges for the current/specific user.
<pre>\$ sudo nano /etc/sudoers</pre>	Directly edit sudoers file, however, editing /etc/sudoers manually can break sudo access if there are syntax errors!
<pre>\$ sudo visudo</pre>	Safely edit sudoers file in the default editor Ahmed All=(ALL) NOPASSWD: /usr/bin/top, /usr/bin/ls » NOPASSWD: ability to run sudo without a password .

Edit Group Permissions

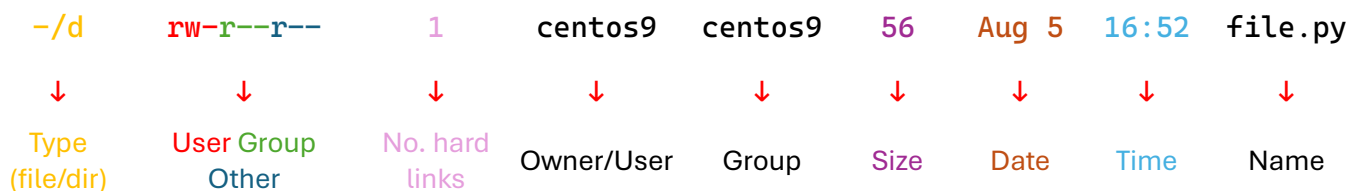
Command	Description
<pre>\$ sudo visudo</pre>	Safely edit group permissions via sudoers file in the default editor. %Marketing ALL=NOPASSWD: /usr/bin/ls, /usr/bin/less, /usr/bin/apt » NOPASSWD: ability to run sudo without a password . <pre>\$ sudo usermod -a -G wheel ahmed \$ sudo gpasswd ahmed wheel</pre>



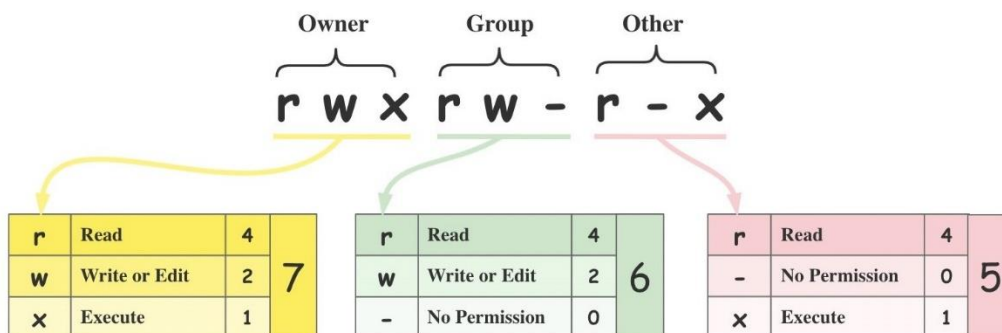
Linux File Permissions

In Linux, file permissions control who can **read**, **write**, or **execute** a file or directory. These permissions help maintain system security and prevent unauthorized access.

```
$ ls -l
total 12
-rw-r--r-- 1 centos9 centos9 56 Aug 5 16:52 file.py
drwxr-xr-x 2 centos9 centos9 4096 Aug 5 16:51 Games
-rw-r--r-- 1 centos9 centos9 56 Aug 5 16:52 hello.txt
```



Binary	Octal	String Representation	Permissions
000	0 (0+0+0)	---	No Permission
001	1 (0+0+1)	--x	Execute
010	2 (0+2+0)	-w-	Write
011	3 (0+2+1)	-wx	Write + Execute
100	4 (4+0+0)	r--	Read
101	5 (4+0+1)	r-x	Read + Execute
110	6 (4+2+0)	rw-	Read + Write
111	7 (4+2+1)	rwX	Read + Write + Execute





Manipulating Files

Change mode of a file

```
$ chmod [u/g/o/a] [+/-/=] [r/w/x] [filename/dirname]
```

Type	Explanation
u	user
g	group
o	other
a	all three

Type	Explanation
+	add permission
-	remove permission
=	set only this permission to u/g/o

Use **-R** flag to apply the permission changes to all files and directories within a specific directory.

Function	Example
Adding Permissions	<pre>\$ chmod a+w file.py \$./file.py \$ chmod u+x file.py \$ chmod -R a+w Games/</pre>
Removing Permissions	<pre>\$ chmod -R a-w Games/ \$ chmod -R a=w Games/ \$ chmod a=wr file.py</pre>
Adding permissions to all (user, group, other users)	<pre>\$ chmod +x file.txt \$ chmod g+w,o-rw,a+x file.txt</pre> » Use comma-separated permissions <u>without spaces</u>.

Changing the owner of the file

```
$ chown Options New_Owner:NEW_GROUP [filename/dirname]
```

Examples
<pre>\$ chown Ahmed file.txt \$ chown :developers file.txt \$ chown Ahmed:developers file.txt \$ chown -R Ahmed Games</pre>



Text Editors (Nano, Vi/Vim)

1) Nano

```
$ nano file.txt
```

If the file exists, nano will open it, otherwise it will create a new file.

Description	Command
Help	CTRL + G
Save	CTRL + S
Save As	CTRL + O
Exit/Close	CTRL + X
Highlight/Select	CTRL + 6
Copy	ALT + 6
Cut	CTRL + K
Paste	CTRL + U
Redo	ALT + E
Undo	ALT + U





2) Vi/Vim

`$ vi file.txt` **or** `$ vim file.txt`

Mode	Key	Description
Command Mode	ESC	- The default mode when you start Vi/Vim. - You use this mode to execute commands for navigating, copying, deleting, and more.
Insert Mode	i	- This is where you can type and edit text. - Press ESC to return to Command Mode.
Visual Mode	v	- Used for highlighting/selecting text. - Press ESC to return to Command Mode.





Function	Key	Mode	Description
Copy (Yank)	y	Visual/Command	Copy the selected text or a specified range.
Paste Text	p	Command Mode	Paste the copied text after the cursor.
Delete Text	d	Visual/Command	Delete the selected text or a specified range.
Delete Line	dd or [num]dd	Command Mode	Delete the current line(s).
Delete Word	dw or d[num]w	Command Mode	Delete a word(s).
Undo Change	u	Command Mode	Undo the last change.
Redo Change	CTRL + r	Command Mode	Redo the change that was undone.
Searching	/<pattern> or ?<pattern>	Command Mode	Start a forward/reverse search for <pattern>.
Move to Line Start	0	Command Mode	Move the cursor to the beginning of the current line.
Move to Line End	CTRL + e	Command Mode	Move the cursor to the end of the current line.
Save and Exit	:wq or ZZ	Command Mode	Save changes and exit Vim.
Exit Without Saving	:q!	Command Mode	Exit Vim without saving changes.
Split Window (Horizontal)	:split <filename>	Command Mode	Open a file in a new horizontal split window.
Split Window (Vertical)	:vsplit <filename>	Command Mode	Open a file in a new vertical split window.
Move Between Splits	CTRL + w + arrow key or h, j, k, l	Command Mode	Move between split wind

Graphical Editors

So far we've learned about various command line editors. However, if you are running Linux as a desktop operating system you might be interested in some graphical text editors and word processors.

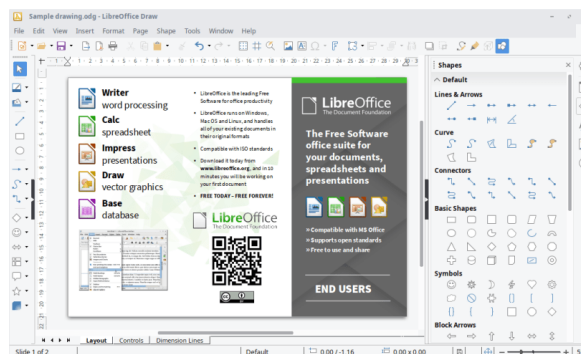
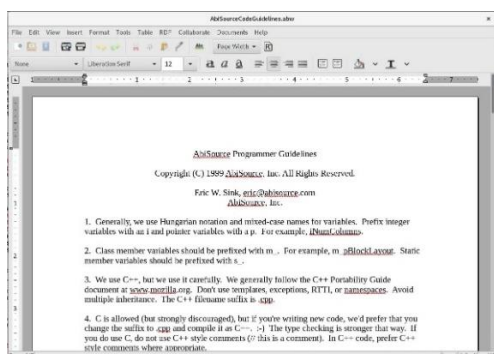
- » **emacs** – Emacs has both graphical and command line modes.
- » **gedit** – The default text editor for the Gnome desktop environment.
- » **gvim** – The graphical version of vim.
- » **kedit** – The default text editor for the [KDE \(K Desktop Environment\)](#) desktop environment.



Microsoft Word Replacement

If you are looking for a Microsoft Word replacement, consider

- [AbiWord](#) or
- [LibreOffice](#) which is a complete office suite with a spreadsheet program, a database, and presentation software.



Code/Text Editors for Linux

- [Visual Studio Code \(VS Code\)](#)
- [Sublime Text](#)
- Gedit – GNOME's simple and lightweight text editor.
- Kate
- etc.

Steps to Set Your Default Editor

You can set `$EDITOR` in your personal initialization files to ensure your favorite editor is used, be it **nano**, **emacs**, **vi/vim**, or something else.

- 1) Open your shell's configuration file (i.e., `~/.bashrc`)

```
$ nano ~/.bashrc
```

- 2) Set the `$EDITOR` variable

```
$ export EDITOR='vim'
```

- 3) Apply the changes

```
$ source ~/.bashrc
```

```
$ echo $EDITOR
```

Now, whenever you invoke a command that requires an editor (e.g., `crontab -e`, `git commit`, etc.), it will open in your preferred editor.



Creating a Collection of Files

If you want to bundle a group of files and/or directories together in an archive (not compressing), you can use the **tar** command which is one of the oldest and common methods for creating and working with compressed backup archives.

- You may want to create a copy or backup of a group of files.
- You may have several files you want to transfer at once or as a set.

```
$ tar [-] c|x|t f <tarfile> <pattern>
```

Note that you have the choice to precede the tar arguments with a hyphen (-) or omit it. For example, `$ tar cf file.tar` is the same as `$ tar -cf file.tar`.

tar option	Description
c	Create a tar archive.
f <file>	The tar archive file to perform operations against.
v	Causes tar to be verbose.
t	Display the table of contents (list).
x	Extract files from the archive.

Examples
<pre>\$ cd /home/MG \$ mkdir data \$ touch data/file.txt data/report.docx main.py \$ tar -cf archived.tar main.py data/ \$ tar -tf archived.tar \$ file archived.tar</pre>
<pre>\$ mkdir /tmp/extract \$ tar -xf archived.tar -C /tmp/extract \$ ls /tmp/extract/data/</pre>
<pre>\$ touch secret secret.bak \$ tar -cvf misc.tar sec* data \$ ls -lh</pre>



Compressing Files To Save Space

Compressing files in Linux is a common way to save space on disk. Linux provides several tools and utilities to compress files and directories effectively.

Command	Description
\$ gzip <file>	Compress file. The resulting compressed file is named file.gz.
\$ gunzip <file>	Uncompress files.
\$ gzcat or zcat	Concatenates and prints compressed files.

Hint: You can use the command `du` to display how much space is used by the file.

```
$ du -h archived.tar
$ gzip archived.tar
$ ls -lh
$ du -h archived.tar.gz
$ gunzip archived.tar.gz
$ ls -lh
```

Linux provides several other tools and utilities to compress files and directories effectively.

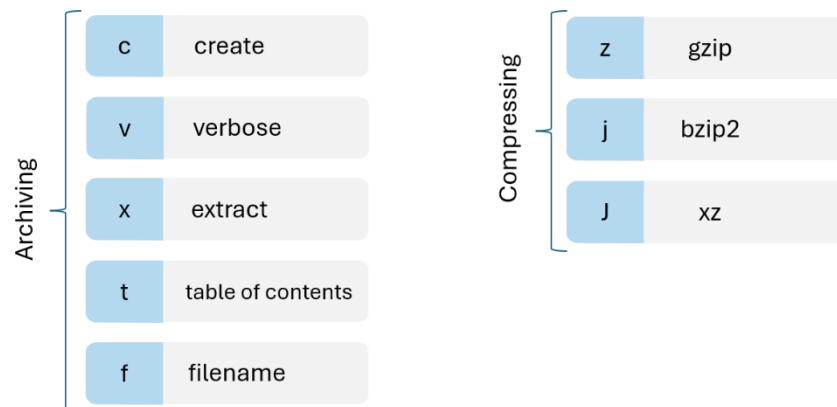
Compression	Decompression
\$ bzip2 <file>	\$ bunzip2 <file>
\$ zip <file>	\$ unzip <file>
\$ xz <file>	\$ xz <file>



Compressing Archives using tar

In modern versions of the **tar** command **gzip** , **bzip2**, and **xz** compressions are built-in.

Type	Compression Percentage	Extension	Compression Command
gzip	75%	.tar.gz	[root@centos9 ~]# tar zcfv file.tar.gz /mnt
bzip2	80%	.tar.bz2	[root@centos9 ~]# tar jcfv file.tar.bz2 /mnt
xz	85%	.tar.xz	[root@centos9 ~]# tar Jcfv file.tar.xz /mnt

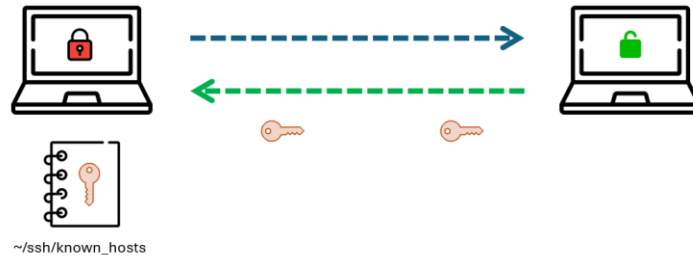


```
$ tar -zcvf compressed_gzip.tar.gz data
$ tar -jcvf compressed_bzip2.tar.bz2 data
$ tar -Jcvf compressed_xz.tar.xz data
```



Connecting remotely to other machines using SSH in Linux?

It's a protocol used to securely run a shell on a remote system. If you have a user account on a remote Linux System providing SSH services, you can remotely log into and run commands on that system.



Command	Usage
<code>sudo systemctl start sshd</code>	Start the SSH service on the system using systemctl.
<code>sudo systemctl stop sshd</code>	Stop the SSH service on a system using systemctl.
<code>sudo systemctl status sshd</code>	Display the status of the SSH service using the service command.
<code>whoami</code>	Display the username of the current user.
<code>ip a</code>	Display all IP addresses and network interfaces.
<code>ssh username@ip_address</code>	Connect to a remote machine using SSH with a specified username and IP address.
<code>exit</code>	Exit the current shell or terminates the SSH session.



Copying Files between Hosts

To copy files between hosts in a Linux environment, you can use several tools, with **scp**, **rsync**, and **sftp** being the most common.

scp	Usage	Strength
	Copy files securely over SSH	Simple, secure, works with individual files
	\$ scp file user@host:/path/to/destination	
rsync	Usage	Strength
	Copy files securely with efficient synchronization	Fast, supports incremental copying
	\$ rsync -avz file user@host:/path/to/destination	
sftp	Usage	Strength
	Interactive file transfer over SSH	Allows navigation and file management
	\$ sftp user@host	

Note: If you are running Windows, you can use the [PuTTY Secure Copy Client](#) (pscp.exe) and the [PuTTY Secure File Transfer](#) client (psftp.exe) programs.

Using scp (Secure Copy)

scp is a command-line tool for securely copying files between local and remote hosts over SSH.

```
$ scp [options] source_file username@destination_host:destination_path
```

▪ Copying a file from local to remote:

```
$ scp /path/to/local/file username@remote_host_ip:/path/to/remote/directory
```

```
$ scp /C/Users/MG/Downloads/file.txt MG@172.26.107.24:/home/MG
```

▪ Copying a file from remote to local:

```
$ scp username@remote_host:/path/to/remote/file /path/to/local/directory
```

```
$ scp MG@172.26.107.24:/home/MG/data.csv /C/Users/MG/Downloads
```



- **Copying a directory recursively (-r flag):**

```
$ scp -r /path/to/local/directory username@remote_host:/path/to/remote/directory  
$ scp -r username@remote_host:/path/to/remote/directory /path/to/local/directory
```

\$ scp -r /C/Users/MG/Downloads/Test MG@172.26.107.24:/home/MG
--

\$ scp -r MG@172.26.107.24:/home/MG/Pictures /C/Users/MG/Downloads
--

- **Using a different SSH port (default port is 22):**

```
$ scp -P port /path/to/local/file username@remote_host:/path/to/remote/directory
```

\$ scp -P 22 MG@172.26.107.24:/home/MG/data.csv /C/Users/MG/Downloads

Other Similar Commands:

- [rsync](#)
- [sftp](#)



Listing Processes and Displaying Information

Process Types:

Running	The process is either executing on CPU or waiting to run.
Sleeping	The process is waiting for an action to be done in order to be running.
Stopped	The process has been Stopped (suspended).
Zombie	A process which is killed but still shows its entry in the process status or the process table is called a zombie process.

- A **process** is a program which is being executed.
- Any process may create a child process.
- All processes are descendants of the first system process, which is system.
- A process can run in the background or foreground.

```
$ echo $$      # See the PID of your current shell process
$ exit
$ echo $$      # Different PID
```

The **ps** (process status) command in Linux is used to list processes and display information about them. It provides various options to customize the output, depending on what information you want. The **ps** command uses **/proc** directory (`$ ls -lh /proc`) to gather information about running processes.

\$ ps [options]

Option	Description
-e	Everything, all processes.
-f	Full format listing.
-u <username>	Display processes running as username.
-p <PID>	Display process information for a PID .
-l	Display detailed information (long format).
aux	Display a comprehensive list of all processes currently running.
...	...

Other Similar Command

\$ pstree	Display running processes in a tree format.
------------------	---



```

mg@192:~$ ps -el | head -n 10
F S  UID      PID     PPID  C  PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
4 S   0         1         0  0   80   0 - 43505 -    ?           00:00:01 systemd
1 S   0         2         0  0   80   0 -    0 -    ?           00:00:00 kthreadd
1 S   0         3         2  0   80   0 -    0 -    ?           00:00:00 pool_workqueue
1 I   0         4         2  0   60 -20 -    0 -    ?           00:00:00 kworker/R-rcu_g
1 I   0         5         2  0   60 -20 -    0 -    ?           00:00:00 kworker/R-rcu_p
1 I   0         6         2  0   60 -20 -    0 -    ?           00:00:00 kworker/R-slub
1 I   0         7         2  0   60 -20 -    0 -    ?           00:00:00 kworker/R-netns
1 I   0         9         2  0   60 -20 -    0 -    ?           00:00:00 kworker/0:0H-events_highpri
1 I   0        10         2  0   80   0 -    0 -    ?           00:00:00 kworker/u512:0-events_unbound

```

- » **F (Flags):** indicates process privileges (e.g., 0 = normal, 4 = superuser).
- » **S (State):** R = Running, S = Sleeping, D = Uninterruptible sleep, Z = Zombie, T = Stopped.
- » **UID (User ID):** the ID of the process owner.
- » **PID (Process ID):** the unique identifier of the process.
- » **PPID (Parent Process ID):** the process that started this one.
- » **C (CPU Usage):** Percentage of CPU usage.
- » **PRI (Priority):** Process priority (lower values mean higher priority).
- » **NI (Nice value):** affects scheduling priority, 0 = default.
- » **ADDR (Address):** Memory address of the process (usually - on modern systems).
- » **SZ (Size):** Process size in memory (in pages).
- » **WCHAN (Waiting Channel):** function where the process is sleeping.
- » **TTY (Teletypewriter):** Terminal associated with the process (e.g., pts/0 for an SSH or terminal session).
- » **TIME:** Total time consumed by the process.
- » **CMD:** Command used to launch the process.

Examples
<pre> \$ ps \$ ps -ef \$ ps -ef head -n 5 \$ ps -fu MG \$ ps -p 10 \$ ps aux head \$ ps aux grep -i vim \$ ls /proc/ \$ ps aux grep 813 # which is shown in /proc </pre>

--- Finding Process IDs ---

Command	Description	Example
<pre> \$ pidof <process name> \$ pgrep <process name> </pre>	Find the PID of a running program.	<pre> \$ pidof vim \$ pgrep vim </pre>



Here're other commands that allow you to view running processes as well, but for real-time monitoring.

--- Real-Time Process Monitoring ---

Option	Description
top	Interactive <u>main processes</u> viewer, for child processes use ps. » Type 1 to display all CPU cores' activity individually. » Type s to change the refresh rate (delay between updates). » Type h for help. » Type k to kill a process by entering its PID. » Type r to renice a process to change its priority (-20 to 19, where lower is higher priority). » Press q to quit. » Type z or Z to toggle color display for better readability. » Type n to change the number of processes shown. » Type O to filter processes based on a specified field (e.g., COMMAND=NAME). » Type Shift+P to sort by CPU usage. » Type Shift+T to sort by process time (CPU time used). » Type Shift+M to by memory usage.
uptime	Display for how long the system has been running.

\$ top

```

MG@centos9:~$ top
top - 20:42:56 up 1:13, 2 users, load average: 2.91, 1.90, 0.87
Tasks: 234 total, 4 running, 230 sleeping, 0 stopped, 0 zombie
%Cpu(s): 68.5 us, 30.9 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.6 hi, 0.0 si, 0.0 st
MiB Mem : 3626.1 total, 1434.3 free, 1399.7 used, 1065.9 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 2226.4 avail Mem

  PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM    TIME+  COMMAND
 6210 MG        20   0   779384   59072   44284 R   29.8   1.6   1:40.84  gnome-terminal-
 7458 MG        20   0   222520    3328    3072 R   29.8   0.1   1:23.79    sh
 4525 MG        20   0   3680408  427580  136752 R   28.3  11.5   2:28.72  gnome-shell
 7244 root        20   0         0         0        0 I    9.0   0.0   0:16.83  kworker/u512:3-events_unbound
 6886 root        20   0         0         0        0 I    1.4   0.0   0:15.98  kworker/u512:1-events_unbound
    1 root        20   0   108304   16716   10816 S    0.0   0.5   0:42.07  systemd
    2 root        20   0         0         0        0 S    0.0   0.0   0:00.01  kthreadd
    3 root        20   0         0         0        0 S    0.0   0.0   0:00.00  pool_workqueue_
    4 root        0 -20         0         0        0 I    0.0   0.0   0:00.00  kworker/R-rcu_g
    5 root        0 -20         0         0        0 I    0.0   0.0   0:00.00  kworker/R-rcu_p
    6 root        0 -20         0         0        0 I    0.0   0.0   0:00.00  kworker/R-slub
    7 root        0 -20         0         0        0 I    0.0   0.0   0:00.00  kworker/R-netns
    9 root        0 -20         0         0        0 I    0.0   0.0   0:00.00  kworker/0:0H-kblockd
   11 root        0 -20         0         0        0 I    0.0   0.0   0:00.00  kworker/R-mm_pe
   13 root        20   0         0         0        0 I    0.0   0.0   0:00.00  rcu_tasks_kthre
   14 root        20   0         0         0        0 I    0.0   0.0   0:00.00  rcu_tasks_rude
   15 root        20   0         0         0        0 I    0.0   0.0   0:00.00  rcu_tasks_trace
   16 root        20   0         0         0        0 S    0.0   0.0   0:00.09  ksoftirqd/0
   17 root        20   0         0         0        0 I    0.0   0.0   0:00.23  rcu_preempt
   18 root        rt    0         0         0        0 S    0.0   0.0   0:00.00  migration/0
   19 root       -51   0         0         0        0 S    0.0   0.0   0:00.00  idle_inject/0
   21 root        20   0         0         0        0 S    0.0   0.0   0:00.00  cpuhp/0
   23 root        20   0         0         0        0 S    0.0   0.0   0:00.00  kdevtmpfs

```



--- Understanding `Top` Attributes ---

PID	The ID of the running process.
USER	The process owner.
PR	The priority of the process.
NI	The nice value of the process.
VIRT	The virtual memory size used by the process.
RES	The resident memory size used by the process (physical memory).
SHR	The amount of shared memory used by the process.
S	The state of the process. » R: Running. » S: Sleeping. » D: Uninterruptible sleep. » Z: Zombie. » T: Stopped or traced. » I: Idle
%CPU	The CPU usage percentage by the process.
%MEM	The memory usage percentage by the process.
TIME+	The total CPU time the process has used since it started.
COMMAND	The name or command that started the process.



Filtering processes by their name (e.g., firefox) by pressing 'o'.

```
MG@centos9:~ - top
top - 20:43:38 up 1:13, 2 users, load average: 2.82, 2.01, 0.95
Tasks: 234 total, 5 running, 229 sleeping, 0 stopped, 0 zombie
%Cpu(s): 68.7 us, 31.0 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.3 hi, 0.0 si, 0.0 st
MiB Mem : 3626.1 total, 1428.6 free, 1405.3 used, 1065.9 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 2220.7 avail Mem
add filter #1 (ignoring case) as: [!]FLD?VAL COMMAND=firefox
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
6210 MG 20 0 779384 59072 44284 R 29.8 1.6 1:53.50 gnome-terminal-
7458 MG 20 0 222520 3328 3072 R 29.8 0.1 1:36.45 sh
4525 MG 20 0 3680408 427580 136752 R 28.1 11.5 2:40.68 gnome-shell
7244 root 20 0 0 0 0 I 5.3 0.0 0:18.85 kworker/u512:3-events unbound
6886 root 20 0 0 0 0 I 4.6 0.0 0:18.39 kworker/u512:1-writeback
5127 MG 20 0 394456 44332 33548 R 0.7 1.2 0:05.32 vmttoolsd
930 root 20 0 383680 8960 7552 S 0.3 0.2 0:05.17 vmttoolsd
7474 MG 20 0 225912 4224 3328 R 0.3 0.1 0:00.10 top
1 root 20 0 108304 16716 10816 S 0.0 0.5 0:42.07 systemd
2 root 20 0 0 0 0 S 0.0 0.0 0:00.01 kthreadd
3 root 20 0 0 0 0 S 0.0 0.0 0:00.00 pool_workqueue_
4 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-rcu_g
5 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-rcu_p
6 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-slub
7 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-netns
9 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/0:0H-kblockd
11 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-mm_pe
13 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_kthre
14 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_rude
15 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_trace
16 root 20 0 0 0 0 S 0.0 0.0 0:00.09 ksoftirqd/0
17 root 20 0 0 0 0 I 0.0 0.0 0:00.23 rcu_preempt
18 root rt 0 0 0 0 S 0.0 0.0 0:00.00 migration/0
```

Toggle color display for better readability by pressing 'z'.

```
MG@centos9:~ - top
top - 21:01:30 up 1:31, 2 users, load average: 2.39, 2.26, 1.94
Tasks: 235 total, 1 running, 234 sleeping, 0 stopped, 0 zombie
%Cpu(s): 1.3 us, 0.3 sy, 0.0 ni, 97.7 id, 0.0 wa, 0.3 hi, 0.3 si, 0.0 st
MiB Mem : 3626.1 total, 1397.2 free, 1437.0 used, 1065.3 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 2189.0 avail Mem
PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
4525 MG 20 0 3648564 398892 136368 S 1.3 10.7 5:04.93 gnome-shell
7565 MG 20 0 225940 4096 3328 R 0.3 0.1 0:00.09 top
1 root 20 0 108304 16716 10816 S 0.0 0.5 0:01.46 systemd
2 root 20 0 0 0 0 S 0.0 0.0 0:00.01 kthreadd
3 root 20 0 0 0 0 S 0.0 0.0 0:00.00 pool_workqueue_
4 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-rcu_g
5 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-rcu_p
6 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-slub_
7 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-netns
9 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/0:0H-kblockd
11 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-mm_pe
13 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_kthre
14 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_rude
15 root 20 0 0 0 0 I 0.0 0.0 0:00.00 rcu_tasks_trace
16 root 20 0 0 0 0 S 0.0 0.0 0:00.09 ksoftirqd/0
17 root 20 0 0 0 0 I 0.0 0.0 0:00.28 rcu_preempt
18 root rt 0 0 0 0 S 0.0 0.0 0:00.00 migration/0
19 root -51 0 0 0 0 S 0.0 0.0 0:00.00 idle_inject/0
21 root 20 0 0 0 0 S 0.0 0.0 0:00.00 cpuhp/0
23 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kdevtmpfs
24 root 0 -20 0 0 0 I 0.0 0.0 0:00.00 kworker/R-inet_
25 root 20 0 0 0 0 S 0.0 0.0 0:00.00 kauditd
27 root 20 0 0 0 0 S 0.0 0.0 0:00.00 khungtaskd
```

...



Running Processes in the Foreground and Background

Up until this point all the commands we have been executing have been running in the foreground. When a command, process, or program is running in the foreground the shell prompt will not be displayed until that process exits.

Why run processes in the background?

- For long running programs it can be convenient to send them to the background.
- Processes that are backgrounded still execute and perform their task, however, they do not block you from entering further commands at the shell prompt.

Command	Description	Example
\$ <command>	Start command in the <u>foreground</u> .	\$ ls -lah
\$ <command> &	Start command in the <u>background</u> .	\$ sleep 1000s &
CTRL + C	<u>Kill</u> the running <u>foreground</u> process.	–
CTRL + Z	<u>Suspend</u> the <u>foreground</u> process.	–
\$ bg [%num]	Background a suspended process.	\$ bg %1
\$ fg [%num]	Foreground a background process.	\$ fg %1
\$ kill [%num]	Kill a process by job number or PID.	\$ kill %1
\$ jobs -l [%num]	List n jobs. » Plus sign (+) : the current job. » Minus sign (-) : the previous job.	\$ jobs -l \$ jobs -l %2

```
MG@centos9:~$ sleep 10000
^Z
[1]+  Stopped                  sleep 10000

MG@centos9 ~]$ bg %1
[1]+  sleep 10000 &

MG@centos9 ~]$ fg %1
sleep 10000
^C
MG@centos9 ~]$
```

```
MG@centos9 ~]$ sleep 1000 &
[1] 7832

MG@centos9 ~]$ kill %1
[1]+  Terminated              sleep 1000

MG@centos9 ~]$ sleep 100000 &
[1] 7846

MG@centos9 ~]$ sleep 200000 &
[2] 7851

MG@centos9 ~]$ jobs
[1]-  Running                  sleep 100000 &
[2]+  Running                  sleep 200000 &
```



Killing Processes

Killing processes in Linux involves terminating processes by sending signals, typically using commands like **kill**, **killall**, or **killall**.

Command	Description
CTRL + C	Kill the running <u>foreground</u> process.
\$ kill <u>-[signal]</u> <PID>	Send a signal to a process by <u>PID</u> .
\$ kill -l	Display a list of signals.
\$ man 7 signal	Display the manual for all signals.
\$ pkill <process name>	Send a signal to process by <u>name</u> .
\$ killall <process name>	Sends a signal to all processes with a specific name.

- » **SIGINT (2)**: Interrupt signal, sent when you press **CTRL+C** in the terminal.
- » **SIGTSTP (20)**: Stop/Suspend a process from the terminal (**Ctrl + Z**).
- » **SIGTERM (15)**: Politely asks a process to terminate gracefully, this is the default signal.
- » **SIGKILL (9)**: Forces termination if a process does not respond.
- » **SIGSTOP (19)**: Stops (pauses) a process, cannot be ignored.
- » **SIGCONT (18)**: Resumes a stopped process.

Running all the following is equivalent which is the default behavior.

```
$ kill <PID>
$ kill -15 <PID>
$ kill -TERM <PID>
$ kill -SIGTERM <PID>
```

```

[MG@centos9 ~]$ sudo tail -f /var/log/messages # Start a dummy process
Dec 5 22:15:18 centos9 systemd[1]: dnf-makecache.service: Deactivated successfully.
Dec 5 22:15:18 centos9 systemd[1]: Finished dnf makecache.
Dec 5 22:25:18 centos9 cupsd[1101]: REQUEST localhost - - "POST / HTTP/1.1" 200 181 Renew-Subscription client-error-not-found
Dec 5 22:46:08 centos9 systemd[1]: Starting Fingerprint Authentication Daemon...
Dec 5 22:46:08 centos9 systemd[1]: Started Fingerprint Authentication Daemon.
Dec 5 22:46:19 centos9 NetworkManager[1077]: <info> [1733431570.4927] agent-manager: agent[1dd77baa26625395,1.67/org.gnome.Shell.NetworkAgent/1000]: agent registered
Dec 5 22:46:38 centos9 systemd[1]: fprintd.service: Deactivated successfully.
Dec 5 22:46:37 centos9 systemd[1]: Starting Fingerprint Authentication Daemon...
Dec 5 22:46:37 centos9 systemd[1]: Started Fingerprint Authentication Daemon.
Dec 5 22:47:27 centos9 systemd[1]: fprintd.service: Deactivated successfully.
Dec 5 22:48:37 centos9 systemd[4230]: Started VTE child process 8128 launched by gnome-terminal-server process 6210.
```

```

[MG@centos9 ~]$ ps aux | grep tail
root      8125  0.0  0.2 239788  956 pts/2    S+   22:48   0:00 sudo tail -f /var/log/messages
root      8127  0.0  0.0 229984  1792 pts/2    S+   22:48   0:00 tail -f /var/log/messages
MG        8164  0.0  0.0 221664  2304 pts/0    S+   22:48   0:00 grep --color=auto tail

[MG@centos9 ~]$ kill -15 8125
[MG@centos9 ~]$
```

```

[MG@centos9 ~]$ sudo tail -f /var/log/messages # Start a dummy process
Dec 5 22:15:18 centos9 systemd[1]: dnf-makecache.service: Deactivated successfully.
Dec 5 22:15:18 centos9 systemd[1]: Finished dnf makecache.
Dec 5 22:25:18 centos9 cupsd[1101]: REQUEST localhost - - "POST / HTTP/1.1" 200 181 Renew-Subscription client-error-not-found
Dec 5 22:46:08 centos9 systemd[1]: Starting Fingerprint Authentication Daemon...
Dec 5 22:46:08 centos9 systemd[1]: Started Fingerprint Authentication Daemon.
Dec 5 22:46:19 centos9 NetworkManager[1077]: <info> [1733431570.4927] agent-manager: agent[1dd77baa26625395,1.67/org.gnome.Shell.NetworkAgent/1000]: agent registered
Dec 5 22:46:38 centos9 systemd[1]: fprintd.service: Deactivated successfully.
Dec 5 22:46:37 centos9 systemd[1]: Starting Fingerprint Authentication Daemon...
Dec 5 22:46:37 centos9 systemd[1]: Started Fingerprint Authentication Daemon.
Dec 5 22:47:27 centos9 systemd[1]: fprintd.service: Deactivated successfully.
Dec 5 22:48:37 centos9 systemd[4230]: Started VTE child process 8128 launched by gnome-terminal-server process 6210.
Terminated
[MG@centos9 ~]$
```



Priority and Nice Values in Linux

Some processes may be highly CPU-intensive but not as important as others and hence can have a lower priority while others may or may not be highly CPU intensive but are very important and hence should have higher priority.

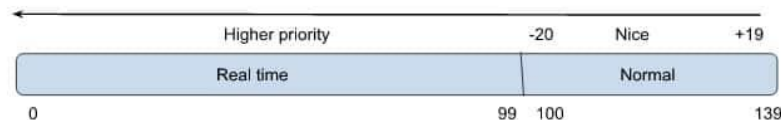
- For example, if there is a process **A**, which detects fraud with input data and there is another process **B**, which makes hourly backups of some data, then the **priority(A) > priority(B)**.
- This ensures that if both **A** and **B** are running at the same time, **A** would be allocated more processing bandwidth.

Priority Value: Determines the actual priority of a process, used by the Linux kernel for scheduling.

- **Range:** 0–139 (real-time: 0–99, user-space: 100–139).

Real-time processes in Linux are high-priority tasks that must complete within strict timing constraints, ensuring they get CPU time before other processes (e.g., embedded systems, telecommunications, ..., etc.).

User processes in Linux are regular tasks initiated by normal users, with lower priority than real-time processes, and their execution can be adjusted using the nice value.



Nice Value: A user-controlled value to influence process priority.

- **Range:** -20 to +19 (-20 is highest priority, 0 is default, +19 is lowest priority)

$$\text{Priority Value} = \text{Nice Value} + 20$$

To see how these work together let us run a process that takes a lot of processing power continuously, we will use a shell script (**infinite.sh**) which uses an infinite loop in it to demonstrate how this works.

```
$ nano infinite.sh
#!/bin/bash
while true; do echo "Running..." > /dev/null 2>&1; done
```

Run `$ sh infinite.sh &` (or `chmod u+x infinite.sh` then `./infinite.sh`) to run the script in the background, use `$ top` then `CTRL+P` to see that this process (PID-3227, for example) is taking high processing power of the CPU.



```

MG@centos9:~ — top
MG@centos9:~
MG@centos9:~ — top
top - 08:29:39 up 32 min, 2 users, load average: 1.77, 0.48, 0.19
Tasks: 228 total, 4 running, 224 sleeping, 0 stopped, 0 zombie
%Cpu(s): 68.1 us, 30.9 sy, 0.0 ni, 0.0 id, 0.0 wa, 1.0 hi, 0.0 si, 0.0 st
MiB Mem : 3626.1 total, 909.5 free, 1212.0 used, 1762.0 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 2414.1 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 2762 MG        20   0  774748  54372 40956 R   30.3    1.5   0:18.91 gnome-terminal-
 3227 MG        20   0  222524   3328  3072 R   30.0    0.1   0:16.09 sh

```

If we run two more processes of **infinite.sh** then all of them (e.g., PID-3227, 3328, 3295) get equal CPU as all have the same priority (e.g., %CPU = 16.9 for all the three).

```

MG@centos9:~ — top
MG@centos9:~
MG@centos9:~
MG@centos9:~ — top
top - 08:31:29 up 34 min, 2 users, load average: 3.25, 1.38, 0.55
Tasks: 232 total, 6 running, 226 sleeping, 0 stopped, 0 zombie
%Cpu(s): 61.5 us, 37.5 sy, 0.0 ni, 0.0 id, 0.0 wa, 0.7 hi, 0.3 si, 0.0 st
MiB Mem : 3626.1 total, 893.7 free, 1227.1 used, 1762.6 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 2398.9 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 2190 MG        20   0 3551284 312816 130216 R   16.9    8.4   1:05.91 gnome-shell
 2762 MG        20   0  775516  55140 40956 R   16.9    1.5   0:49.75 gnome-terminal-
 3227 MG        20   0  222524   3328  3072 R   16.9    0.1   0:46.94 sh
 3295 MG        20   0  222524   3328  3072 R   16.9    0.1   0:02.71 sh
 3328 MG        20   0  222524   3328  3072 R   16.9    0.1   0:01.14 sh

```

Now let us give these processes different **nice** values, there are two ways to do this:

- 1) Start a new process with a nice value using the **\$ nice** command

```
$ nice -n <nice_val> [command]
```

```
$ nice -n 10 sh infinite.sh &
```

- » When we run this we get another process (PID-8509) with the priority as 30 (Priority = 20 + 10 = 30).
- » As it has the least priority, it gets the least amount of CPU.

```

MG@centos9:~ — top
MG@centos9:~
MG@centos9:~
MG@centos9:~ — sh infinite.sh
MG@centos9:~ — top
top - 23:55:59 up 4:26, 2 users, load average: 5.23, 2.94, 1.39
Tasks: 240 total, 7 running, 233 sleeping, 0 stopped, 0 zombie
%Cpu(s): 53.3 us, 36.8 sy, 8.9 ni, 0.0 id, 0.0 wa, 0.7 hi, 0.3 si, 0.0 st
MiB Mem : 3626.1 total, 1408.0 free, 1419.5 used, 1072.5 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used. 2206.6 avail Mem

  PID USER      PR  NI   VIRT   RES   SHR  S  %CPU  %MEM     TIME+ COMMAND
 8509 MG        30  10  222520   3328  3072 R   14.3    0.1   0:49.15 sh
 8585 MG        20   0  222520   3328  3072 R   14.3    0.1   0:35.67 sh
 8626 MG        20   0  222520   3456  3200 R   14.3    0.1   0:33.12 sh
 8668 MG        20   0  222520   3328  3072 R   14.3    0.1   0:04.79 sh
 4525 MG        20   0 3648856 400624 136752 R   14.0   10.8   7:19.55 gnome-shell
1103 root        20   0   16620   9216   7936 S    0.0    0.2   0:00.02 sshd
4650 MG        20   0   58728 25036 19328 S    0.0    0.7   0:00.23 gnome-shell-cal
5092 MG        20   0   67840 15352  9600 S    0.0    0.4   0:00.05 gsd-sharing
7343 MG        20   0   224504 6016  3712 S    0.0    0.2   0:00.08 bash
8514 MG        20   0   224112 5632  3712 S    0.0    0.2   0:00.01 bash

```



2) Change a running process nice value by its **PID** using `$ renice` command

```
$ renice -n <nice_val> -p [PID]
```

```
$ renice -n 5 -p 8585
```

- » When we run this, the process with PID-8585 get its priority changed to 25 (Priority = 20 + 5 = 25).
- » The CPU is allocated accordingly as well.

```
top - 00:10:10 up 4:40, 2 users, load average: 5.97, 5.63, 4.03
Tasks: 243 total, 7 running, 236 sleeping, 0 stopped, 0 zombie
%Cpu(s): 45.2 us, 37.2 sy, 17.3 ni, 0.0 id, 0.0 wa, 0.3 hi, 0.0 si, 0.0 st
MiB Mem : 3626.1 total, 1384.8 free, 1431.9 used, 1090.3 buff/cache
MiB Swap: 2048.0 total, 2048.0 free, 0.0 used, 2194.2 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR  S  %CPU  %MEM    TIME+  COMMAND
 4525 MG        20   0 3648856 400516 136752 R   14.2   10.8   8:39.84  gnome-shell
 8509 MG        30  10 222520  3328  3072 R   14.2    0.1   2:56.11  sh
 8668 MG        20   0 222520  3328  3072 R   14.2    0.1   2:11.75  sh
 8585 MG        25   5 222520  3328  3072 R   13.9    0.1   2:42.63  sh
 8626 MG        20   0 222520  3456  3200 R   13.9    0.1   2:40.08  sh
 1103 root         20   0 16620  9216  7936 S    0.0    0.2   0:00.02  sshd
```

Note that:

- » Regular users can only set positive nice values (0 to 19) to lower a process's priority.
- » Only the root user (administrator) can set negative nice values (-20 to -1) to increase a process's priority.

--- Summary ---

Feature	nice	renice
When used	When starting a new process	For changing running processes
Root privilege	Required for <u>negative nice values</u>	
Effect	Sets the initial nice value	Modifies an existing nice value



Analyzing and Sorting Logs

In Linux, system logs are stored in the **/var/log** directory, which contains log files for various system activities, applications, services, and security. Analyzing and sorting logs in Linux is a crucial step for troubleshooting, monitoring, and understanding the behavior of the system.

```
[MG@centos9 ~]$ ls -lhF /var/log
total 5.2M
drwxr-xr-x. 2 root root 4.0K Nov 29 14:11 anaconda/
drwx----- 2 root root 23 Aug 1 15:42 audit/
-rw----- 1 root root 0 Dec 6 00:00 boot.log
drwxr-x--- 2 chrony chrony 6 Nov 11 11:44 chrony/
-rw----- 1 root root 21K Dec 6 00:01 cron
drwxr-xr-x. 2 lp sys 57 Oct 1 16:25 cups/
-rw-r----- 1 root root 50 Dec 4 11:57 firewalld
drwx--x--x. 2 root gdm 6 Jul 26 21:36 gdm/
-rw-r--r-- 1 root root 1.1K Dec 5 23:21 hawkey.log
-rw-rw-r-- 1 root utmp 286K Dec 5 19:10 lastlog
-rw----- 1 root root 0 Nov 29 14:04 maillog
-rw----- 1 root root 4.4M Dec 6 00:16 messages
drwx----- 2 root root 6 Nov 29 14:02 private/
drwx----- 3 root root 17 Oct 22 13:20 samba/
-rw----- 1 root root 94K Dec 6 00:15 secure
drwx----- 2 root root 6 Aug 10 2021 speech-dispatcher/
-rw----- 1 root root 0 Nov 29 14:04 spooler
drwxr-x--- 2 sssd sssd 26 Oct 21 16:17 sssd/
-rw----- 1 root root 0 Nov 29 14:02 tallylog
drwxr-xr-x. 2 root root 23 Aug 8 00:36 tuned/
-rw-rw-r-- 1 root utmp 30K Dec 5 19:10 wtmp
```

Log File	Purpose
/var/log/messages	Most syslog messages.
/var/log/secure	Log file for security and authentication-related messages and errors.
/var/log/maillog	Log file with main server-related messages.
/var/log/boot.log	Messages related to system start up.
...	...

```
$ df -h # report file system disk space usage
```

```
$ cat /var/log/messages
$ less /var/log/boot.log
$ tail -f /var/log/messages
$ grep "error" /var/log/messages
$ sort /var/log/messages
$ awk '{print $1, $2, $3}' /var/log/ messages
```



Scheduling Jobs

Cron is a time-based job scheduling service that automates tasks or performs routine maintenance. It checks for scheduled jobs every minute and executes them accordingly.

- **cron:** the service responsible for running scheduled jobs, typically started when the system boots.
- **Crontab (Cron Table):** a program used to create, read, update, and delete job schedules. The crontab file defines when and what commands to run, with five time specification fields (minutes, hour, day of the month, month of the year, and day of the week), followed by the command to execute.

Crontab Format

```
* * * * * command
| | | | |
| | | | +-- Day of the Week (0-6)
| | | +---- Month of the Year (1-12)
| | +----- Day of the Month (1-31)
| +----- Hour (0-23)
+----- Minute (0-59)
```

Commands	
\$ crontab -e	Edit (e) the crontab file for the current user.
\$ crontab -l	List (l) current user's crontab entries.
\$ crontab -r	Remove (r) the current user's crontab file.

Examples ([Crontab Generator](#)):

```
# Run every Monday at 07:00.
0 7 * * 1 /opt/sales/bin/weekly-report
```

```
0 7 * * 1 /opt/sales/bin/weekly-report
| | | | |
| | | | +-- Day of the Week (0-6)
| | | +---- Month of the Year (1-12)
| | +----- Day of the Month (1-31)
| +----- Hour (0-23)
+----- Minute (0-59)
```

```
0 0 * * 3 <exec_file> # Cron job every Wednesday at midnight
* /30 * * * ls -l # Cron job every half hour
0 0 * * 1-5 # Run on weekdays only at midnight
```

Example
<pre>\$ crontab -e # open crontab file * * * * * rm /home/mg/.local/share/Trash/files/* \$ sudo tail -f /var/log/messages</pre>

Once you write and save a crontab entry using **\$ crontab -e**, the cron daemon (cron) automatically schedules the job based on the specified timing.

```
$ sudo systemctl status crond
```



Package Management on CentOS, Fedora, and RedHat-based Distributions (Installing Software)

A **package** in Linux is a collection of files that make up an application, along with metadata such as the app's description, version, and dependencies. To install or remove packages, superuser privileges are required.

A **package manager** is used to install, upgrade, and remove packages, while also handling dependencies and tracking installed packages and their versions.

1) Using yum Package Manager

The **yum (Yellowdog Updater, Modified)** command is a package management tool for RPM-based distributions like CentOS, Fedora, and RedHat.



Search for a package

```
$ yum search <package name>
```

```
$ yum search inkscape
```

Install a package

```
$ sudo yum install <package name>
```

```
$ sudo yum install inkscape
```

Remove a package

```
$ sudo yum remove <package name>
```

```
$ sudo yum remove inkscape
```

Get information about a package

```
$ yum info <package name>
```

```
$ yum info inkscape
```

List all installed/available Packages

```
$ sudo yum list available|installed
```

```
$ sudo yum list installed
```

Check for available updates

```
$ sudo yum check-update
```

```
$ sudo yum check-update
```

Update a Specific Package

```
$ sudo yum update <package name>
```

```
$ sudo yum update nginx
```

Update All System Packages

```
$ sudo yum update
```

```
$ sudo yum update
```



2) Using dnf Package Manager

The **dnf** (**D**andified **YUM**) package manager is an improved, modern version of **yum**, designed to address some limitations of **yum** while maintaining its core functionality.

3) Using rpm Package Manager (Low-Level)

The **rpm** (**R**ed**H**at **P**ackage **M**anager) command can also be used to manage packages, such as installing a package from a file, and listing files belonging to a package, ..., etc.

Search for a Package
<code>\$ rpm -qi <package name></code>

```
$ rpm -qi SpiderOak
```

Get information about a Package
<code>\$ rpm -qi <package name></code>

```
$ rpm -qi SpiderOak
```

Download an RPM Package
<code>\$ dnf download <package name></code>

```
$ dnf download nano
```

Install a Package
<code>\$ rpm -ivh <package.rpm></code>

```
$ sudo rpm -ivh nano-*.rpm
```

List all installed Packages
<code>\$ rpm -qa</code>

```
$ rpm -qa | sort | head
```

Update a Specific Package
<code>\$ rpm -Uvh <package.rpm></code>

```
$ sudo rpm -Uvh update nginx
```

Remove a package
<code>\$ rpm -e <package name></code>

```
$ sudo rpm -e SpiderOak
```

List all files belonging to a package
<code>\$ rpm -ql <package name></code>

```
$ rpm -ql inkscape
```

The **RPM** package manager itself does not handle package updates or dependency resolution. It is a low-level package manager that installs, removes, and queries individual .rpm packages but does not manage repositories or system-wide updates.

For updating packages, you need to use **DNF** (or **YUM**). However, if you still want to use RPM, you would need to manually download and install updated packages.



--- yum vs. dnf vs. rpm ---

Feature	yum (Legacy)	dnf (Modern)	rpm (low-level)
Default in	CentOS 7 and older	CentOS 8, 9 (Stream), RHEL 8+	All RHEL-based distros
Performance	Slower, higher memory usage	Faster, lower memory usage	Very fast, but lacks dependency resolution
Dependency Resolution	Uses older algorithms	More efficient dependency solving	No automatic dependency resolution
Plugin System	Relies on plugins for many features	Built-in support for advanced features	No plugin support
Package Caching	Stores metadata in /var/cache/yum/	Uses sqlite3 for faster processing	No package caching
Parallel Downloads	Not supported	Supported (improves speed)	N/A
Command Syntax	\$ yum install package	\$ dnf install package (same syntax)	\$ rpm -ivh package.rpm

Example: <https://code.visualstudio.com/Download>

